Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1:

Laboratory Work 1: Concurrency in Practice

Elaborated:

Poiata Calin, gr. FAF-243
Covali Ilie

Verified: asist. univ. Cristofor Fistic

Chișinău – 2026

Contents

1 Objective and Requirements **2**
1.1 Laboratory Objective 2
1.2 Hardware and Runtime Environment 2
1.3 Dataset Generation 2

2 Task 1a: Direct File Partitioning and Multithreaded Sign Counting **4**
2.1 Methodology and Boundary Handling 4
2.2 Experimental Results 4
2.3 Task 1a Questions and Discussion 5

3 Task 1b: Three Synchronization Strategies for Sign Counting **6**
3.1 Experimental Setup 6
3.2 Experimental Results 6
3.3 Task 1b Questions and Discussion 7

4 Task 1c: Producer and Consumers with Bounded Queue **8**
4.1 Architecture and Specification 8
4.2 Experimental Results 8
4.3 Task 1c Questions and Discussion 9

5 Task 2: Many Files on Slow Storage (USB Flash Drive) **10**
5.1 Experimental Setup and Cache Eviction 10
5.2 Experimental Results 10
5.3 Task 2 Questions and Discussion 11

6 Conclusion **12**

7 Control Questions **12**

1 Objective and Requirements

1.1 Laboratory Objective

The objective of this laboratory work is to investigate real-world multithreaded concurrency in systems programming, measuring how thread counts and workload partitioning strategies influence execution time, parallel speed-up, and peak memory utilization. Key goals include:

1. Observing CPU-bound workload scaling across physical cores and logical processors.
2. Identifying parallel speed-up saturation points and context-switching penalties.
3. Demonstrating race conditions and non-deterministic memory corruption in unsynchronized concurrent updates.
4. Quantifying the performance overhead of mutual exclusion (locks) compared to thread-local aggregation.
5. Implementing and profiling a producer–consumer pipeline using a thread-safe bounded blocking queue.
6. Analyzing I/O-bound workload behavior on slow removable storage (USB flash drive) with cold OS page cache eviction.

1.2 Hardware and Runtime Environment

To ensure true OS-level thread concurrency without the Global Interpreter Lock (GIL), the implementation uses the newly released **free-threaded CPython 3.13** runtime (`cpython-3.13t`), managed via the `uv` package and virtual environment manager.

Table 1: Hardware and Environment Specifications

Component	**Specification**
CPU Model	AMD Ryzen 7 4700U with Radeon Graphics
Architecture	x86_64 (Zen 2)
Physical Cores	8
Logical Processors	8 (1 thread per core, SMT disabled)
RAM	16 GB DDR4
Primary Storage (Disk)	SAMSUNG MZALQ512HALU-000L1 NVMe SSD (512 GB)
Removable Storage (USB)	Ventoy USB 3.0 Flash Drive (32 GB, partition `/dev/sda1`)
Operating System	Arch Linux (Kernel 7.2.11-arch1-1)
Programming Language	Python 3.13.16+freethreaded (Clang build)
GIL Status	Disabled (`sys._is_gil_enabled() == False`)
Environment Tool	`uv` 0.13+

1.3 Dataset Generation

The input dataset consists of a text file `numbers.txt` containing exactly 50 000 000 uniformly distributed pseudo-random integers in the range $[-1\,000\,000, 1\,000\,000]$, formatted with one

number per line. The file size is approximately 352.33 MB.

$$\text{Total} = \text{Count}(\text{negative}) + \text{Count}(\text{positive}) + \text{Count}(\text{zero}) = 50\,000\,000 \tag{1}$$

For Task 2, `numbers.txt` is split, in sequential order, into 8 distinct partition files (`part_0.txt` through `part_7.txt`) containing exactly 6 250 000 numbers each, copied directly to the USB flash drive. Every task must satisfy this strict count conservation invariant.

2 Task 1a: Direct File Partitioning and Multithreaded Sign Counting

2.1 Methodology and Boundary Handling

Task 1a evaluates parallel file reading and counting across thread configurations $T \in \{1,2,4,8,16,32,64\}$. Reading the entire file on a single thread or pre-loading the text into RAM is prohibited. The file of size S bytes is partitioned into T disjoint nominal byte intervals:

$$\text{Range}_t = \left[\left\lfloor \frac{t \cdot S}{T} \right\rfloor, \left\lfloor \frac{(t+1) \cdot S}{T} \right\rfloor \right), \quad t \in \{0,1,\ldots,T-1\} \tag{2}$$

Because nominal byte offsets arbitrarily split text lines, a boundary reconciliation algorithm is implemented:

- **Initial Seek:** Thread $t > 0$ seeks to byte offset $\text{start}_t - 1$ and inspects the preceding character. If the preceding character is a newline (`\n`), the thread begins parsing immediately at start_t. If it is not a newline, the thread discards the remainder of the fractional line via `readline()` and begins parsing from the next complete line.
- **Termination Boundary:** Threads $t < T-1$ continue reading lines until their byte counter satisfies $\text{curr_pos} \geq \text{end}_t$. The fractional line spanning across end_t is fully consumed by thread t, ensuring that the subsequent thread $t+1$ (which discards up to that line's newline) will neither skip nor duplicate any record.
- **Tail Processing:** Thread $T-1$ reads continuously until EOF.

2.2 Experimental Results

The benchmark was executed with 2 separate runs per thread configuration using isolated subprocesses to capture exact peak resident set size (Peak RSS).

Table 2: Task 1a Benchmark Results (50 000 000 integers, NVMe SSD)

Threads	Run 1 (ms)	Run 2 (ms)	Avg Time (ms)	Speed-up	Peak RAM (MB)	Total Counted
1	5479.4	5441.6	5460.5	1.00×	35.7	50 000 000
2	4268.9	4277.1	4273.0	1.28×	37.9	50 000 000
4	2252.1	2409.3	2330.7	2.34×	41.8	50 000 000
8	1572.5	1437.1	1504.8	3.63×	52.1	50 000 000
16	1602.5	1668.5	1635.5	3.34×	64.3	50 000 000
32	1499.4	1515.4	1507.4	3.62×	62.2	50 000 000
64	1588.6	1524.1	1556.4	3.51×	60.4	50 000 000

Correctness verification: Across all thread configurations (1 through 64), every trial produced identical sign totals:

$$\text{Negative} = 25\,002\,492, \quad \text{Positive} = 24\,997\,491, \quad \text{Zero} = 17, \quad \text{Total} = 50\,000\,000 \tag{3}$$

This confirms that the boundary-seeking logic operates with zero omission and zero duplication.

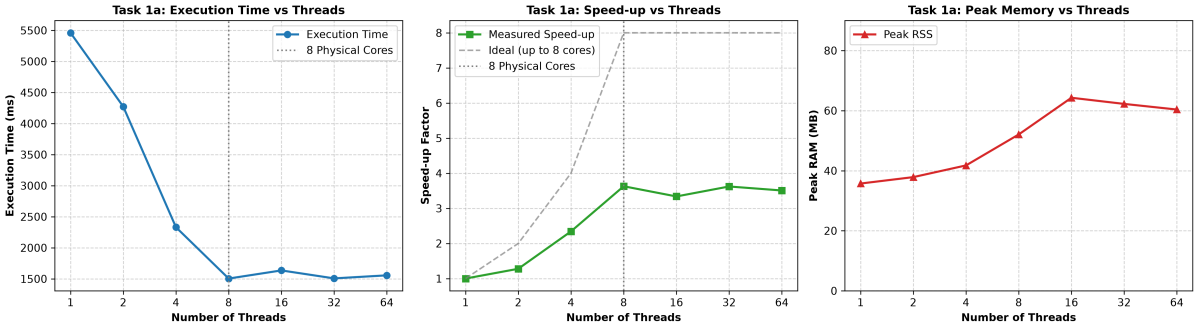


Figure 1: Task 1a: Execution time, speed-up, and peak memory versus thread count.

2.3 Task 1a Questions and Discussion

1. **At which number of threads does the speed-up stop growing? Relate it to your physical cores and logical processors:**
   The speed-up grows substantially from 1 thread (5460.5 ms) to 8 threads (1504.8 ms, achieving a 3.63× speed-up), but stops growing beyond 8 threads. This correlates exactly with the test machine's hardware: the AMD Ryzen 7 4700U has 8 physical cores and 8 logical processors (without SMT). Because there are 8 hardware execution pipelines, assigning more than 8 worker threads provides no additional computational capacity.

2. **Why do 32 and 64 threads not help? Are they slower, and why?**
   Threads 32 and 64 do not improve throughput (1507.4 ms and 1556.4 ms vs 1504.8 ms at 8 threads). When 64 threads contend for 8 CPU cores, the OS kernel must continuously preempt and context-switch threads. Context switching incurs CPU cycle penalties for saving/restoring register states, invalidates L1/L2 instruction and data caches, and causes cache line bouncing. The workload is strictly bounded by the 8 physical CPU cores.

3. **Does the peak memory depend on the number of threads? Why?**
   Peak memory remains low across all configurations, increasing only slightly from 35.7 MB (1 thread) to 64.3 MB (16 threads). This modest growth occurs because each OS thread allocates its own execution stack and small internal CPython I/O buffer (8 KB per file descriptor). Because each thread streams lines sequentially from disk rather than buffering the file in memory, memory consumption is independent of the dataset size (352 MB).

3 Task 1b: Three Synchronization Strategies for Sign Counting

3.1 Experimental Setup

In Task 1b, all 50 000 000 integers are preloaded into a contiguous in-memory 32-bit signed integer buffer using Python's native `array.array('i')`. The array occupies 190.73 MB of contiguous RAM.

With thread count matched to the machine's 8 logical execution processors ($T = 8$), the elements are partitioned into 8 uniform slices of 6 250 000 integers. The signs are counted under three distinct synchronization paradigms:

1. **Version 1 (Unsynchronized Shared Counters):** All 8 threads concurrently execute in-place increments (`shared.neg += 1`, `shared.pos += 1`, `shared.zero += 1`) without locks or atomics.

2. **Version 2 (Lock-Protected Counters):** Every counter update is enclosed within a mutual exclusion lock (`threading.Lock()`).

3. **Version 3 (Thread-Local Counters):** Each thread independently counts signs in local CPU registers/variables. Totals are summed into a global accumulator only after all threads finish.

3.2 Experimental Results

Version 1 was executed three consecutive times to observe race condition non-determinism.

Table 3: Task 1b Benchmark Results ($T = 8$ threads, 50 000 000 in-memory integers)

Synchronization Version	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Avg Time (ms)	Counted
Version 1 (Unsynchronized)	26322.2	27398.5	27488.2	27069.6	20 733 102
Version 2 (Lock-Protected)	68286.4	—	—	68286.4	50 000 000
Version 3 (Thread-Local)	2836.1	—	—	**2836.1**	50 000 000

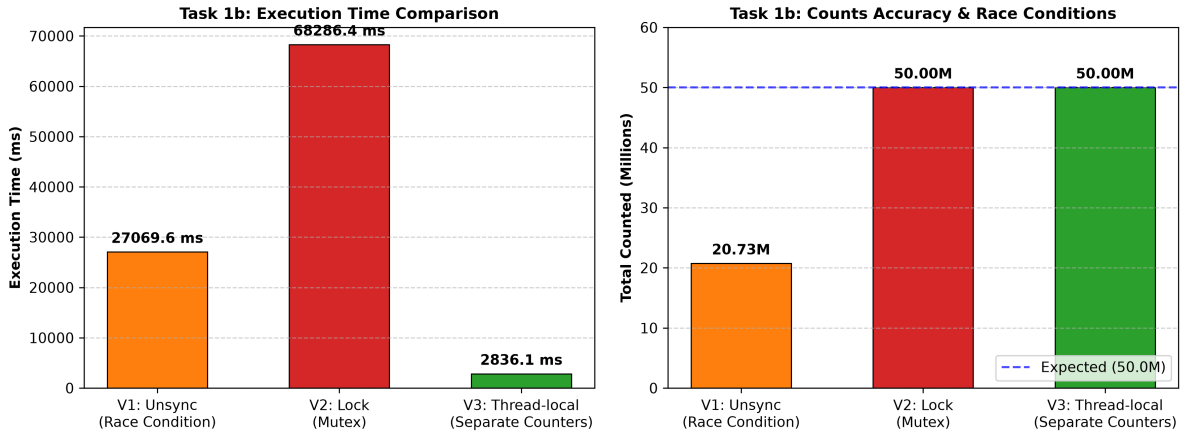


Figure 2: Task 1b: Execution time and total numbers counted across the three synchronization approaches.

3.3 Task 1b Questions and Discussion

1. **Rank the three versions of Task 1b by time. Which of them are correct? Explain each result:**
   The execution time ranking from fastest to slowest is:

$$\begin{aligned}\text{Version 3 (Thread-Local, 2836.1 ms)} &< \text{Version 1 (Unsynchronized, 27069.6 ms)} \\ &< \text{Version 2 (Lock-Protected, 68286.4 ms)} \quad (4)\end{aligned}$$

   Only **Version 2** and **Version 3** are correct (50 000 000 numbers counted). Version 1 is incorrect, losing approximately 58.5% of all updates due to rampant uncoordinated concurrent writes across all 8 CPU cores.

2. **Explanation of Version 1 (Unsynchronized):**
   Under free-threaded Python (no GIL), `count += 1` translates to a non-atomic read-modify-write sequence: (1) load counter from memory into register, (2) increment register, (3) store register back to memory. When multiple threads execute this sequence concurrently on different CPU cores, thread interleaving causes stale reads and overwritten writes.

3. **Explanation of Version 2 (Lock-Protected):**
   Version 2 guarantees correctness by serializing access with a mutex lock, but it is **24.08× slower** than Version 3 (68286.4 ms vs 2836.1 ms). Acquiring and releasing a lock 50 000 000 times with 8 threads contending generates severe lock contention, bus locking, and operating system sleep/wakeup syscalls (futexes).

4. **Explanation of Version 3 (Thread-Local):**
   Version 3 achieves optimal throughput because each core increments private variables residing in fast CPU registers or L1 cache with zero synchronization overhead. Combining the counts requires only a trivial $O(T)$ post-join addition.

4 Task 1c: Producer and Consumers with Bounded Queue

4.1 Architecture and Specification

Task 1c implements an asynchronous pipeline separating file reading (I/O) from data parsing and sign counting (CPU):

- **Producer (Reader Thread):** Exactly one dedicated thread reads `numbers.txt` line by line in binary mode, groups them into batches of 10 000 lines, and enqueues them into a thread-safe blocking queue (`queue.Queue(maxsize=100)`).
- **Bounded Queue:** Holds at most 100 batches (1 000 000 lines total). When the queue reaches capacity, the producer blocks until consumers make space, guaranteeing bounded memory usage.
- **Consumers (Worker Threads):** Evaluated across $W \in \{1, 2, 4, 8, 16, 32, 64\}$ workers. Each worker repeatedly dequeues a batch, parses the integers, and maintains local counters.
- **Termination Protocol:** At EOF, the producer pushes W distinct sentinel values (`None`) into the queue. Each consumer shuts down upon receiving a sentinel.

4.2 Experimental Results

Table 4: Task 1c Benchmark Results (1 Producer, Bounded Queue maxsize=100, batch=10 000)

Workers	Time (ms)	Speed-up	Peak RAM (MB)	Total Counted	Negative	Positive
1	8106.2	1.00×	586.7	50 000 000	25 002 492	24 997 491
2	7969.4	1.02×	586.7	50 000 000	25 002 492	24 997 491
4	7955.5	1.02×	586.7	50 000 000	25 002 492	24 997 491
8	8096.0	1.00×	586.7	50 000 000	25 002 492	24 997 491
16	8194.0	0.99×	586.7	50 000 000	25 002 492	24 997 491
32	7966.5	1.02×	586.7	50 000 000	25 002 492	24 997 491
64	7885.0	1.03×	586.7	50 000 000	25 002 492	24 997 491

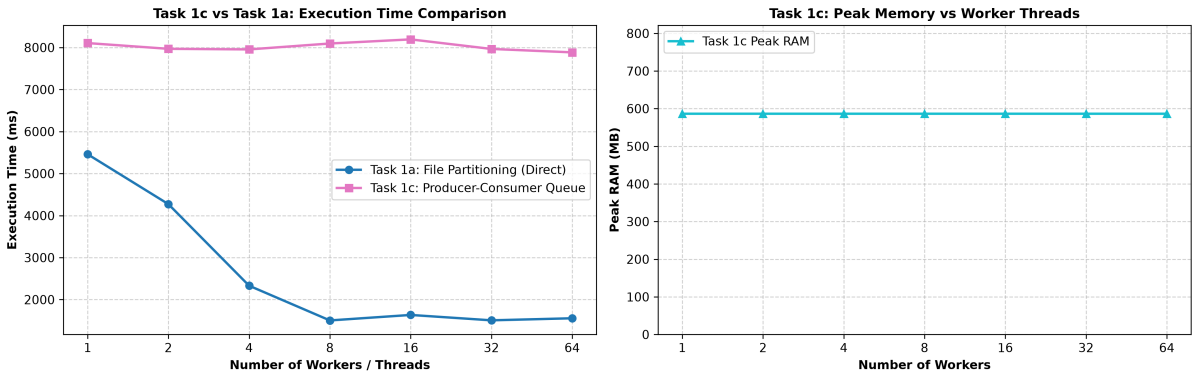


Figure 3: Task 1c: Producer–Consumer execution time compared against Task 1a, and peak memory versus worker count.

4.3 Task 1c Questions and Discussion

1. **Compare Task 1c with Task 1a. Which one is faster, and why?**
   Task 1a is significantly faster than Task 1c. At 8 threads, Task 1a completes in 1504.8 **ms**, whereas Task 1c requires 8096.0 **ms**—making Task 1a **5.38× faster**.

   - In Task 1a, all 8 threads read and parse their independent file chunks in parallel directly from NVMe storage.
   - In Task 1c, the architecture is bottlenecked by the single sequential reader thread. The producer must read all 50 000 000 lines, allocate 5 000 Python list objects containing 10 000 line references each, and coordinate through queue locks and condition variables.

2. **At which number of workers does Task 1c stop getting faster?**
   Task 1c stops getting faster immediately at **1 worker**. Across all worker counts ($W \in \{1, 2, 4, 8, 16, 32, 64\}$), execution time remains flat at ≈ 8.0 seconds. Because the consumers can parse a 10 000-line batch much faster than the single reader can produce it, the queue is perpetually empty and the workers spend the majority of their time idling in wait states.

3. **What would happen with batches of a single line?**
   Batching single lines would require 50 000 000 separate enqueue (`q.put()`) and dequeue (`q.get()`) operations. Each operation requires acquiring an internal mutex, signaling condition variables, and modifying queue nodes. This extreme lock contention and synchronization overhead would cause execution time to explode by orders of magnitude (hundreds of seconds).

4. **What would happen with a queue that has no size limit?**
   An unbounded queue would allow the fast producer to read the entire file and push all 50 000 000 lines into memory before consumers finish. Allocating 50 000 000 line objects and list nodes would consume several gigabytes of RAM, destroying the memory isolation guarantee of the system.

5 Task 2: Many Files on Slow Storage (USB Flash Drive)

5.1 Experimental Setup and Cache Eviction

Task 2 investigates how concurrency behaves when the workload transitions from fast NVMe storage to slow removable media:

- **Storage Medium:** Ventoy USB 3.0 Flash Drive (32 GB, formatted with FAT32).
- **Workload Partitioning:** The 50 000 000 numbers dataset is split sequentially into 8 distinct files of 6 250 000 numbers each (`part_0.txt` to `part_7.txt`, ≈ 44.0 MB each).
- **Unit of Work:** A file is the atomic unit of work. Each thread reads its assigned file in a **single operation into one byte buffer** via `f.read()`, parses the buffer in-memory, and tallies the signs.
- **Cold-Cache Enforcement:** To ensure no data is served from the operating system's page cache (RAM), the USB filesystem is unmounted and remounted before every benchmark run using `udisksctl unmount` and `udisksctl mount`. Additionally, `os.posix_fadvise(POSIX_FADV` is called on the file descriptors.

5.2 Experimental Results

The benchmark was evaluated across 1, 2, and 4 threads dividing the 8 files evenly (8 files/thread, 4 files/thread, and 2 files/thread respectively).

Table 5: Task 2 Benchmark Results on USB Flash Drive (Cold OS Page Cache)

Threads	Files / Thread	Time (ms)	Speed-up	Peak RAM (MB)	Total Counted
1	8	18 276.0	1.00×	400.0	50 000 000
2	4	22 200.8	**0.82× (Slowdown)**	777.5	50 000 000
4	2	22 715.7	**0.80× (Slowdown)**	1539.3	50 000 000

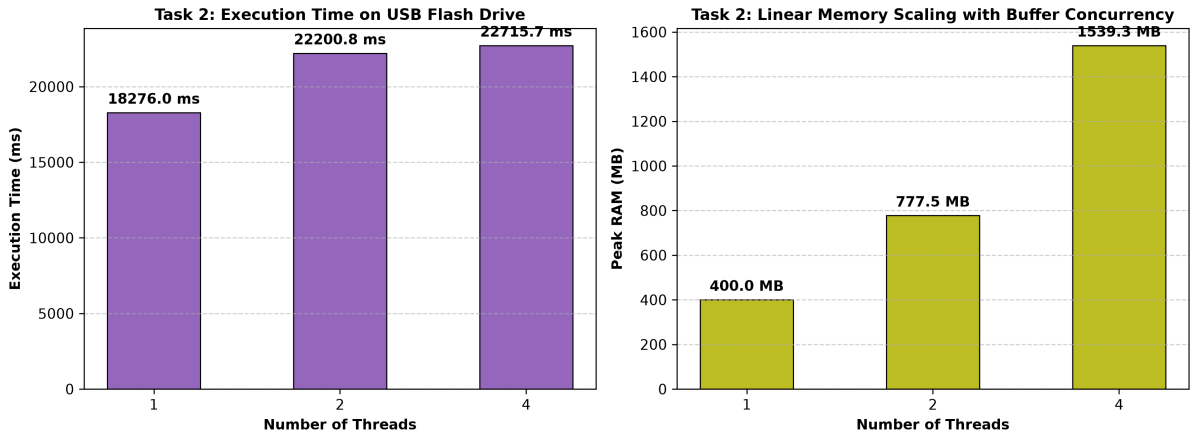


Figure 4: Task 2: Execution time and linear peak memory scaling on USB flash drive.

5.3 Task 2 Questions and Discussion

1. **Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound? Support the answer with numbers:**
   On 1 thread:

   - Task 1a (NVMe SSD): 5460.5 **ms**
   - Task 2 (USB Drive): 18276.0 **ms**

   Parsing the identical 50 000 000 numbers on the USB drive is **3.35× slower** (12.82 seconds of pure storage transfer penalty). On the NVMe SSD, data transfer was instantaneous, making CPU parsing the bottleneck (CPU-bound). On the USB flash drive, the transfer rate of the USB controller and NAND flash is the bottleneck (18.28 s total), demonstrating that the workload is strictly **I/O-bound**.

2. **On the USB drive, do more threads help? Why not, when lecture showed that I/O-bound work benefits from concurrency?**
   On the USB drive, more threads **do not help**—in fact, concurrency causes a performance degradation (**0.80× slowdown**):

   - **The Nature of USB Storage:** A standard USB flash drive operates over a half-duplex USB bus and utilizes a cheap flash controller lacking multi-queue capabilities (unlike NVMe with dozens of submission queues).
   - **Random vs. Sequential I/O:** When 1 thread reads 8 files sequentially, the flash controller streams continuous blocks efficiently. When 2 or 4 threads issue concurrent reads to different files, the requests interleave, forcing the flash controller to jump across physical NAND flash blocks. This creates head-of-line blocking, internal write-buffer thrashing, and bus contention.
   - **Why Network I/O Differs:** Concurrency benefits I/O-bound workloads when waiting on *independent, distributed remote channels* (e.g., thousands of separate web servers across a network). On a single shared local physical bus, concurrent requests create physical device contention.

3. **Linear Memory Scaling Explanation:**
   Peak RAM scales linearly with thread count (400.0 MB → 777.5 MB → 1539.3 MB). Because each file of 6 250 000 numbers is loaded in a single operation into one byte buffer and parsed via `splitlines()`, one active file occupies ≈ 390 MB of memory structures. With 4 threads operating simultaneously, 4 entire files are buffered concurrently in RAM (4×390 MB ≈ 1.54 GB).

6 Conclusion

Through systematic benchmarking across four distinct tasks on free-threaded Python 3.13, this laboratory work provided concrete empirical insights into multithreaded systems programming:

1. **CPU-Bound Workload Characterization:** Task 1a on fast NVMe storage demonstrated classic CPU-bound parallel scaling. Scaling scaled from 1 to 8 threads (3.63× speed-up) perfectly aligned with the 8 physical CPU cores of the Ryzen 7 4700U, after which speed-up saturated due to lack of additional hardware execution units.

2. **True Race Conditions in Free-Threaded Python:** Task 1b confirmed that without the Global Interpreter Lock, unsynchronized shared memory writes produce severe lost updates ($\approx 58.5\%$ data loss with 8 threads). Lock synchronization fixed correctness but incurred a 24× performance penalty due to extreme lock contention. Thread-local aggregation proved to be the optimal design pattern, delivering both 100% correctness and maximum throughput.

3. **Producer Bottlenecks in Pipelines:** Task 1c demonstrated that a concurrent consumer pool cannot overcome a single-threaded sequential producer bottleneck. The producer-consumer model must be designed with parallel producers or chunked multi-file ingestion to avoid worker starvation.

4. **I/O-Bound Workload on Removable Storage:** Task 2 demonstrated that I/O-bound workloads on single-controller storage devices (USB flash drives) suffer from thread concurrency. Interleaving read requests destroyed sequential transfer efficiency, causing execution time to increase from 18.28 s to 22.72 s.

7 Control Questions

1. **What is the difference between a physical core and a logical processor? Where did it show in your charts?**
   A physical core is an independent, complete hardware processing unit fabricated onto the silicon die, equipped with its own ALUs, FPUs, registers, and L1/L2 caches. A logical processor is an architectural execution context exposed to the OS scheduler (e.g., via Simultaneous Multithreading / Intel Hyper-Threading, where one physical core hosts two execution contexts sharing the same underlying hardware). On the AMD Ryzen 7 4700U, there are 8 physical cores and 8 logical processors (no SMT). In Figure 1, this manifested as a steep speed-up increase from 1 to 8 threads, followed by a flat horizontal plateau from 8 to 64 threads.

2. **How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?**
   Using preceding-byte boundary inspection: each thread $t > 0$ seeks to its nominal byte offset $\text{start}_t - 1$ and inspects the preceding byte. If the preceding byte was a newline (`\n`), the cut landed cleanly on a line boundary and the thread starts immediately. If not, the cut sliced through the middle of a line; the thread discards the remainder of the fractured line via `readline()` and begins parsing from the next complete line. Concurrently, the

preceding thread $t-1$ reads past end_{t-1} until it encounters a newline, ensuring that the fractured line is processed exactly once.

3. **Explain step by step how unsynchronised shared counters lose updates.**
   The statement `shared.neg += 1` consists of three separate machine operations:

   (a) **Read:** Core 0 and Core 1 both read the counter variable from shared memory into their respective private registers (both read value V).

   (b) **Modify:** Both cores independently increment their local register to $V+1$.

   (c) **Write:** Core 0 writes $V+1$ back to memory. Immediately after, Core 1 writes $V+1$ back to memory.

   Two distinct increments occurred, but the final stored value only increased by 1. One increment was permanently lost. Across 50 000 000 increments across 8 threads, this race condition destroyed over 29.2 million counts (58.5%).

4. **Why are per-thread counters faster than shared counters protected by a lock?**
   When multiple cores update a shared locked variable, they must acquire a mutual exclusion primitive. This triggers cache coherence invalidation traffic (MESI protocol): the cache line holding the lock and counter must bounce between CPU core caches, causing high bus latency and CPU stalls. If the lock is held, contending threads are suspended into kernel sleep states (futexes). Per-thread counters operate strictly on thread-local registers or L1 cache lines with zero bus communication, zero locks, and zero pipeline stalls.

5. **What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?**
   "I/O-bound" means the execution speed of the process is limited by the rate at which input/output operations complete rather than the CPU's processing power. For a USB drive, the bottleneck is a single shared physical controller and half-duplex serial bus; concurrent threads issue competing read requests that interleave and cause random seek/block fragmentation on the same device. For a network server, the bottleneck is high round-trip network latency waiting on thousands of *independent* remote endpoints; while thread A waits on client A's network packet, the CPU can immediately switch to thread B and process client B's packet, achieving massive concurrency overlap.

6. **What is the difference between dividing the work into equal parts in advance and taking it from a queue? When is each one better?**
   Static partitioning (in advance): Divides the workload into predetermined static chunks (S/T). It has zero synchronization overhead during execution, minimal memory footprint, and is optimal when work items have uniform processing times and the storage medium supports parallel access (e.g., Task 1a on NVMe).
   Dynamic queue (producer–consumer): Workers pull dynamic batches from a shared thread-safe queue. It naturally balances workloads when work items have unpredictable or highly variable processing times, but incurs continuous synchronization, queue locking, and memory allocation overhead.